Project Execution Prompt

You are tasked with completing this project from its current state into a **research-grade Adaptive Learning Platform (ALP)** suitable for publication.

Before doing anything else, **load and use the** `Adaptive Learning Research Architect` **skill** located in this repository. Treat that skill as your primary operating instructions throughout the project. All planning, implementation, experimentation, documentation, and validation must follow the standards and workflow defined by the skill.

The skill is not optional. It overrides your default behavior whenever there is a conflict.

Objective

Your objective is to build a complete Adaptive Learning Platform capable of answering the following research question:

> **Do Adaptive Learning Systems that use deep behavioral analysis improve learning efficiency compared to traditional adaptive learning systems?**

The goal is not simply to make the software work.

The goal is to produce a complete research platform that can generate statistically valid results suitable for an academic research paper.

Existing Project

The repository already contains an initial implementation.

Current workflow:

```
Student

↓

Enter Topic

↓

LLM generates question bank
```

```
↓

Questions categorized by difficulty

↓

Student answers question

↓

Behavioral features collected

↓

Rule-based mastery engine

↓

Next question selected

↓

Repeat until mastery threshold
```

Do **not** discard the existing implementation.

Instead:

- Understand the architecture.
- Review every file.
- Identify incomplete components.
- Improve the existing implementation.
- Preserve compatibility whenever possible.

First Tasks

Before writing any code:

1. Load the Adaptive Learning Research Architect skill.
2. Read the entire repository.
3. Explain the current architecture.
4. Identify incomplete modules.
5. Produce a development roadmap.

6. List assumptions.
7. Identify technical debt.
8. Identify research gaps.
9. Only after planning should implementation begin.

Never immediately start coding.

Repository Review

The first deliverable should include:

- High-Level Architecture
- Low-Level Design
- Database Schema
- API Documentation
- Current Data Flow
- Current Adaptive Logic
- Missing Features
- Scalability Concerns
- Research Limitations

After the review, create a prioritized implementation plan.

Required Development Stages

Follow the development stages defined in the loaded skill.

Do not skip stages.

The expected progression is:

1. Improve Rule-Based Adaptive Engine
2. Improve Behavioral Feature Engineering
3. Student Simulator
4. Synthetic Dataset Generation
5. Machine Learning Pipeline
6. Model Comparison Framework
7. Dynamic Learning Roadmap
8. Experimental Evaluation
9. Research Paper

Complete one stage before progressing to the next.

Behavioral Features

The frontend already collects:

- Response Time
- Reading Time
- Post Interaction Time
- Correctness
- Question Difficulty
- Number of Attempts
- Question Skip
- Previous Correctness
- Option Changes
- Mouse Distance
- Mouse Speed
- Hover Time
- Typing Speed
- Backspaces
- Delete Frequency
- Typing Pause Duration
- Question Number
- Session Duration
- Accuracy Decay

Do not remove these features.

Instead, improve how they are used.

Latent Variables

The system should infer:

- Knowledge
- Confidence
- Fatigue
- Engagement
- Cognitive Load

These latent variables should become the central representation of student state.

Rule-Based Engine

Improve the existing rule-based engine.

Avoid arbitrary heuristics.

Every rule should be:

- Explainable
- Documented
- Justified
- Modular
- Testable

The rule engine should estimate latent variables and recommend adaptive learning actions.

Student Simulator

Develop a realistic student simulator capable of generating large-scale synthetic interaction data.

Support multiple learner archetypes.

Each simulated learner should possess independent behavioral characteristics such as:

- Learning Rate
- Confidence
- Guess Probability
- Reading Speed
- Typing Speed
- Fatigue Growth
- Motivation
- Persistence
- Memory Retention
- Attention Span

The simulator should produce realistic interaction sequences.

Dataset Generation

Generate datasets suitable for machine learning.

Each record should contain:

- Student State
- Behavioral Features
- Question Metadata
- Rule-Based Outputs

- Learning Outcome
- Selected Learning Action

Support reproducible dataset generation.

Machine Learning

Train and compare multiple models.

Required baselines:

- Logistic Regression
- Decision Tree
- Random Forest

Required advanced models:

- XGBoost
- LightGBM
- CatBoost

Required neural model:

- Multi-Layer Perceptron

Optional:

- LSTM
- Transformer

Benchmark every model.

Never stop after one successful model.

Evaluation

Compare:

- Static Curriculum
- Existing Rule-Based ALP
- Improved Rule-Based ALP
- ML-Based ALP

Evaluate using:

- Accuracy
- Precision
- Recall
- F1
- ROC-AUC
- Training Time
- Inference Time
- Feature Importance
- Learning Efficiency
- Time to Mastery
- Questions Required for Mastery

Perform statistical significance testing where appropriate.

Dynamic Learning Roadmap

Extend the platform from next-question prediction to adaptive curriculum generation.

Represent knowledge as a graph of concepts and prerequisite relationships.

Continuously update each learner's roadmap based on observed behavior and inferred knowledge state.

Documentation

Throughout development, continuously update:

- Architecture Documentation
- API Documentation
- Dataset Documentation
- Experimental Methodology
- Model Documentation
- Research Notes

Do not postpone documentation until the end.

Research Paper

Maintain the research paper as development progresses.

The final paper should include:

- Abstract
- Introduction
- Literature Review
- Related Work
- Methodology
- Behavioral Feature Engineering
- Rule-Based Engine
- Student Simulation
- Dataset Generation
- Machine Learning Models
- Experimental Design
- Results
- Statistical Analysis
- Discussion
- Limitations
- Future Work
- Conclusion
- References

Engineering Standards

Follow the standards defined by the loaded skill.

Specifically:

- Modular architecture
- Clean code
- Loose coupling
- High cohesion
- Comprehensive documentation
- Reproducible experiments
- Versioned datasets
- Versioned models
- Automated testing
- Benchmarking
- Explainable algorithms

Working Style

Do not wait for user approval after every small task.

Autonomously complete each stage while documenting assumptions and important design decisions.

Whenever a decision has multiple reasonable alternatives:

1. Evaluate the alternatives.
2. Explain the trade-offs.
3. Choose the strongest option.
4. Document the reasoning.

Continuously improve previous implementations if better approaches are discovered.

Definition of Done

The project is complete only when all of the following have been produced:

- Fully functional Adaptive Learning Platform
- Enhanced Rule-Based Personalization Engine
- Deep Behavioral Analytics Module
- Student Simulator
- Synthetic Dataset Generator
- Machine Learning Pipeline
- Comparative Model Evaluation
- Dynamic Personalized Learning Roadmap
- Experimental Framework
- Statistical Analysis
- Complete Documentation
- Complete Research Paper

Do not stop until every deliverable has been implemented, validated, documented, and integrated into the project.